

Lab 10 Spring WebFlux & WebClient

รายวิชา: CP353002 หลักการออกแบบและพัฒนาซอฟต์แวร์

เรื่อง: Reactive Programming, Spring WebFlux, Mono, Flux และ WebClient

ชื่อ-นามสกุล: นายพัชรพล กองแก้ว

รหัสนักศึกษา: 673380415-5

Section: 4

1. Reactive Programming vs Blocking

Blocking Programming เป็นการทำงานที่ Thread ต้องรอให้กระบวนการหนึ่งทำงานเสร็จก่อนจึงสามารถทำงานต่อได้ เช่น การรอข้อมูลจาก Server หรือ Database ทำให้ระหว่างรอ Thread ไม่สามารถนำไปทำงานอื่นได้

Reactive Programming เป็นการทำงานแบบ Asynchronous และ Non-blocking โดย Thread ไม่จำเป็นต้องรอผลลัพธ์ เมื่อข้อมูลพร้อมจึงดำเนินการต่อผ่าน Reactive Stream ทำให้สามารถรองรับ Request จำนวนมากได้อย่างมีประสิทธิภาพมากขึ้น

2. Mono vs Flux

`Mono<T>` ใช้สำหรับข้อมูลที่มีผลลัพธ์ 0 หรือ 1 ค่า เหมาะสำหรับการทำงานที่คืนข้อมูลเพียงรายการเดียว เช่น การค้นหาสินค้าตาม ID การบันทึกสินค้า หรือการดูราคาหลังส่วนลด

ตัวอย่าง

Java

```
public Mono<Product> findById(String id) {  
    Product product = store.get(id);  
  
    if (product == null) {  
        return Mono.empty();  
    }  
  
    return Mono.just(product);  
}
```

`Flux<T>` ใช้สำหรับข้อมูลที่มีผลลัพธ์ 0 ถึงหลายค่า เหมาะสำหรับการดึงข้อมูลหลายรายการ เช่น การดูสินค้าทั้งหมด หรือการค้นหาสินค้าตาม Category

ตัวอย่าง

Java

```
public Flux<Product> findAll() {  
    return Flux.fromIterable(store.values());  
}
```

ดังนั้น หากต้องการผลลัพธ์เพียงรายการเดียวจะใช้ `Mono` แต่หากต้องการผลลัพธ์หลายรายการจะใช้ `Flux`

3. WebClient

WebClient ใช้สำหรับเรียก HTTP API แบบ Reactive และ Non-blocking โดยสามารถรับผลลัพธ์เป็น Mono หรือ Flux ได้ ตัวอย่าง

Java

```
public Mono<Product> getProductById(String id) {  
    return client.get()  
        .uri("/products/{id}", id)  
        .retrieve()  
        .bodyToMono(Product.class);  
}
```

คำอธิบาย Method Chain

.get()

กำหนด HTTP Method ที่ต้องการใช้เป็น GET เพื่อร้องขอข้อมูลจาก Server

.uri()

กำหนด URL หรือ Endpoint ที่ WebClient ต้องการเรียก เช่น

Java

```
.uri("/products/{id}", id)
```

.retrieve()

ใช้รับ HTTP Response ที่ส่งกลับมาจาก Server เพื่อนำ Response ไปประมวลผลต่อ

.bodyToMono()

ใช้แปลง Response Body ให้เป็น Mono<T> เมื่อคาดว่าจะได้รับข้อมูลเพียงหนึ่งรายการ

Java

```
.bodyToMono(Product.class)
```

.bodyToFlux()

ใช้แปลง Response Body ให้เป็น Flux<T> เมื่อคาดว่าจะได้รับข้อมูลหลายรายการ

Java

```
.bodyToFlux(Product.class)
```

การใช้ WebClient ในการทดลองนี้ไม่มีการเรียก .block() จึงยังคงเป็นการทำงานแบบ Non-blocking

4. Code ของ Repository, Service และ Controller

4.1 ProductRepository

ProductRepository ทำหน้าที่จัดเก็บและเข้าถึงข้อมูลสินค้า โดยใช้ ConcurrentHashMap เป็น In-memory Storage

Java

```
// ค้นหาสินค้าตาม ID
// คืน Mono<Product> เนื่องจากพบสินค้าได้ 0 หรือ 1 รายการ
public Mono<Product> findById(String id) {
    Product product = store.get(id);

    if (product == null) {
        return Mono.empty();
    }

    return Mono.just(product);
}

// ดึงสินค้าทั้งหมด
// ใช้ Flux เพราะสามารถมีสินค้าได้หลายรายการ
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

// บันทึกสินค้าและคืนสินค้าที่บันทึกเป็น Mono<Product>
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// ลบสินค้าตาม ID และคืน Mono<Void>
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

// ค้นหาสินค้าตาม Category
// ใช้ filter() เพื่อเลือกเฉพาะสินค้าที่ Category ตรงกัน
public Flux<Product> findByCategory(String category) {
    return findAll()
        .filter(p ->
            p.getCategory().equalsIgnoreCase(category));
}
```

4.2 ProductService

ProductService ทำหน้าที่จัดการ Business Logic และเป็นตัวกลางระหว่าง Controller และ Repository

Java

```
// ค้นหาสินค้าตาม ID
// หากไม่พบจะส่ง Error ผ่าน Reactive Stream
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(
            Mono.error(
                new RuntimeException(
                    "Product not found: " + id
                )
            )
        );
}

// ดึงสินค้าทั้งหมด
public Flux<Product> getAll() {
    return repository.findAll();
}

// บันทึกสินค้า
// หากไม่มี ID จะสร้าง UUID ก่อนบันทึก
public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(
            java.util.UUID.randomUUID().toString());
    }
    return repository.save(product);
}

// ลบสินค้าตาม ID
public Mono<Void> delete(String id) {
    return repository.deleteById(id);
}

// ค้นหาสินค้าตาม Category
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// ใช้ map() แปลง Mono<Product> เป็น Mono<Double>
// เพื่อคืนราคาหลังส่วนลด
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(Product::getDiscountedPrice);
}
```

4.3 ProductController

ProductController ทำหน้าที่รับ HTTP Request และเรียกใช้งาน ProductService โดยคืนค่า Mono หรือ Flux โดยตรง ไม่มีการใช้ .block()

Java

```
// GET /products/{id} ค้นหาสินค้าตาม ID
@GetMapping("/{id}")
public Mono<Product> getById(@PathVariable String id) {
    return service.getById(id);
}

// GET /product ดูสินค้าทั้งหมด
@GetMapping
public Flux<Product> getAll() {
    return service.getAll();
}

// POST /products เพิ่มสินค้าใหม่
@PostMapping
public Mono<Product> save(@RequestBody Product product) {
    return service.save(product);
}

// DELETE /products/{id} ลบสินค้าตาม ID
@DeleteMapping("/{id}")
public Mono<Void> delete(@PathVariable String id) {
    return service.delete(id);
}

// GET /products/category/{category}
// ค้นหาสินค้าตาม Category
@GetMapping("/{category}/{category}")
public Flux<Product> getByCategory(
    @PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price
// ดูราคาสินค้าหลังส่วนลด
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(
    @PathVariable String id) {

    return service.getDiscountedPrice(id);
}
```